

DE L'ORDRE DES REGISTRES

*Les collecteurs d'impôts de Reymerswale tiennent leur registre à jour, ligne après ligne :
une base de données spatiale fait de même, mais interrogeable en un instant sur des
millions de lignes à la fois.*



FIG. V. MARINUS VAN REYMERWALE, « LES
COLLECTEURS D'IMPÔTS », VERS 1540

Index spatial GiST, requêtes et jointures spatiales, topologie, performance, PostGIS
Raster, exposition web d'une base spatiale.

SOMMAIRE

1. PERFORMANCE : EXPLAIN ANALYZE ET LE PLAN DE REQUÊTE SPATIAL

2. RASTER EN BASE : POSTGIS RASTER

3. PASSER À L'ÉCHELLE : PARTITIONNEMENT ET PARALLÉLISATION

4. SÉCURITÉ ET CONTRÔLE D'ACCÈS SUR UNE BASE SPATIALE PARTAGÉE

Le module Le Compas introduit PostGIS en une section : une base de données spatiale existe, elle sait exécuter une requête spatiale en SQL. Cette salle va plus loin sur ce qui rend une base spatiale réellement utilisable à grande échelle : comment un index spatial fonctionne concrètement, comment lire le plan d'exécution d'une requête, comment imposer une cohérence topologique, et comment une base spatiale s'insère dans le reste d'une infrastructure (raster, web, historique). Trois pistes complètes ci-dessous (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin.

VOIR AUSSI : AVANT DE COMMENCER : POSTGIS, UNE PREMIÈRE REQUÊTE SPATIALE

Le Compas introduit PostGIS et une première requête ST_Intersects — prérequis direct de cette salle.

APPROFONDISSEMENT

1. PERFORMANCE : EXPLAIN ANALYZE ET LE PLAN DE REQUÊTE SPATIAL

Face à une requête spatiale lente, la première étape de diagnostic n'est jamais de deviner mais de lire le plan d'exécution réel que PostgreSQL a choisi, via `'EXPLAIN ANALYZE'`. Ce plan révèle en particulier si l'index spatial a été utilisé (« Index Scan using idx_parcelles_geom ») ou ignoré au profit d'un balayage complet de table (« Seq Scan »), la cause la plus fréquente d'une requête spatiale anormalement lente sur une grande table.

EXEMPLE

POURQUOI UN INDEX EXISTANT N'EST PARFOIS PAS UTILISÉ

PostgreSQL peut ignorer un index spatial existant si les statistiques internes de la table sont obsolètes (résolu par `'ANALYZE nom_table'`), ou si la requête compare la géométrie à une expression calculée plutôt qu'à une valeur directe (ex. reprojeter une colonne à la volée dans la clause WHERE empêche souvent l'utilisation de l'index sur cette colonne) — un piège classique, la reprojection doit être faite une fois, en amont, pas répétée à chaque ligne comparée dans la requête.

APPROFONDISSEMENT

2. RASTER EN BASE : POSTGIS RASTER

Au-delà du vecteur, l'extension PostGIS Raster stocke des données raster (une image satellite, un MNT) directement en base, découpées automatiquement en tuiles internes pour rester interrogeables efficacement, avec des fonctions dédiées (`'ST_Value'` pour lire la valeur d'un pixel à une coordonnée donnée, `'ST_SummaryStats'` pour des statistiques de zone directement en SQL) — la même logique que les statistiques de zone du module Le Compas, mais exécutée en base plutôt que dans un logiciel SIG desktop.

APPROFONDISSEMENT

3. PASSER À L'ÉCHELLE : PARTITIONNEMENT ET PARALLÉLISATION

Au-delà de quelques dizaines de millions de lignes, même un index GiST bien utilisé peut ne plus suffire : le partitionnement spatial découpe une table en plusieurs partitions physiques (par exemple par région ou par dalle géographique), chacune interrogeable indépendamment. PostgreSQL peut alors ignorer entièrement les partitions dont la zone ne recoupe pas la requête (partition pruning), sans même consulter leur index, une économie que l'index seul n'apporte pas sur une table monolithique gigantesque.

REMARQUE

PARTITIONNER N'EST PAS GRATUIT NON PLUS

Un découpage trop fin multiplie le nombre de partitions à gérer (maintenance, statistiques séparées) sans gain supplémentaire ; un découpage trop grossier laisse des partitions individuellement trop volumineuses pour bénéficier du pruning. Le bon découpage dépend de la distribution réelle des requêtes attendues (par région administrative ? par grille régulière ?), pas d'une règle universelle — la même logique de compromis que le choix d'un h en KDE (module Les Statistiques) ou d'une résolution de grille en analyse raster.

À TOI DE VOIR

À TOI DE VOIR

Une base nationale de parcelles cadastrales (plusieurs dizaines de millions de lignes) reçoit presque exclusivement des requêtes limitées à un seul département à la fois. Propose un critère de partitionnement adapté à cet usage, et explique en quoi il réduirait le travail réel de PostgreSQL par rapport à un index GiST seul sur la table entière.

APPROFONDISSEMENT

4. SÉCURITÉ ET CONTRÔLE D'ACCÈS SUR UNE BASE SPATIALE PARTAGÉE

Une base spatiale accessible à plusieurs applications et plusieurs équipes (voir section 2, jointures exécutées directement en SQL) pose une question qu'un fichier isolé ne pose jamais : qui a le droit de lire, modifier ou supprimer quoi. PostgreSQL gère ces droits au niveau du rôle (utilisateur ou groupe), de la table, et jusqu'à la ligne individuelle — trois granularités bien distinctes, souvent combinées.

MÉCANISME	GRANULARITÉ	EXEMPLE D'USAGE GÉOSPATIAL
GRANT / REVOKE	Table ou colonne entière	Un compte de lecture seule pour le service web (SELECT uniquement), aucun droit d'écriture
Row-Level Security (RLS)	Ligne individuelle, selon une politique SQL	Un agent de terrain ne voit que les parcelles de son secteur géographique, filtrées par une politique fondée sur ST_Intersects
Schémas séparés	Ensemble de tables	Isoler les données de production des tables de travail/import, sans dupliquer la base entière

EXEMPLE

LA RLS, UN FILTRE SPATIAL APPLIQUÉ AUTOMATIQUEMENT

Une politique Row-Level Security peut restreindre chaque requête à une zone géographique sans que l'application cliente n'ait jamais besoin d'ajouter elle-même la condition spatiale : ``CREATE POLICY secteur_agent ON parcelles USING (ST_Intersects(geom, secteur_de(current_user)))``. La politique s'applique automatiquement à toute requête du rôle concerné, y compris une jointure ou une vue construite par-dessus — un agent malveillant ou un bug applicatif ne peut pas contourner la restriction en écrivant une requête différente.

ATTENTION

UN RÔLE APPLICATIF UNIQUE MASQUE QUI A FAIT QUOI

Faire se connecter toute une application par un seul rôle PostgreSQL partagé (pratique courante pour simplifier la configuration) rend impossible de savoir, a posteriori, quel utilisateur final a modifié quelle géométrie — l'historisation par trigger (section 5, versionner une donnée spatiale) enregistre alors « qui » au sens du rôle technique, pas de la personne réelle. Propager l'identité de l'utilisateur final jusqu'à la base (rôle nommé, ou colonne dédiée renseignée par l'application) est nécessaire dès qu'une traçabilité réglementaire (cadastre, urbanisme) est en jeu.

- Bilan — à retenir : EXPLAIN ANALYZE révèle si un index est réellement utilisé (Index Scan) ou ignoré (Seq Scan) ; PostGIS Raster applique la logique des statistiques de zone directement en base, en tuiles internes ; au-delà de quelques dizaines de millions de lignes, le partitionnement spatial permet d'ignorer entièrement des partitions non concernées, un gain que l'index seul n'apporte plus à cette échelle ; GRANT/REVOKE, Row-Level Security et schémas séparés gèrent l'accès à trois granularités différentes, un rôle applicatif unique partagé casse toute traçabilité individuelle.

VOIR AUSSI : VOIR AUSSI : WMS, WMTS, WFS

Le module Cartographie web détaille comment cette base, une fois optimisée, est exposée côté client.